

MVC الدرس 3: المتحكمات ونمط

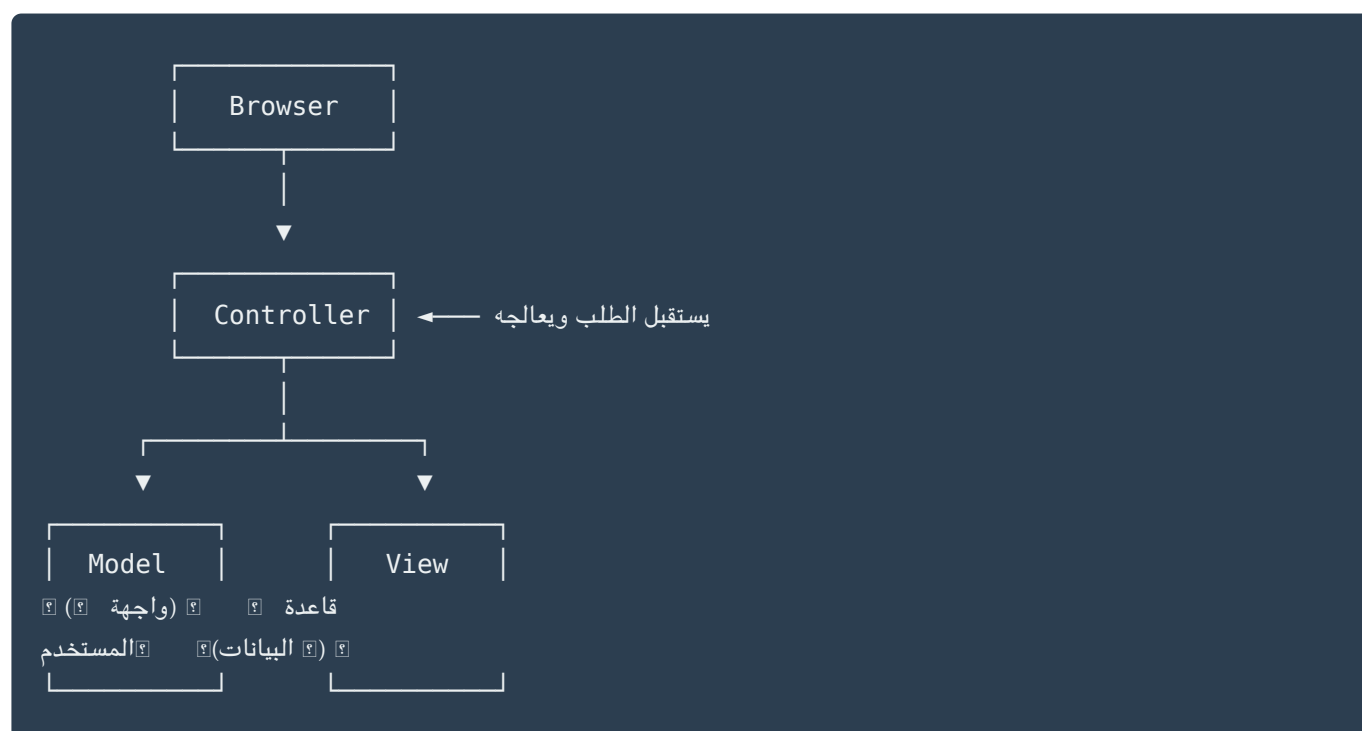
جدول المحتويات

1. MVC مقدمة في نمط
2. ما هي المتحكمات؟
3. إنشاء المتحكمات
4. أنواع المتحكمات
5. Resource Controllers
6. حقن التبعية
7. Middleware في المتحكمات
8. التمارين العملية

MVC مقدمة في نمط

MVC؟ ما هو نمط

النموذج-العرض-المتحكم)، وهو نمط معماري لتنظيم الكود (Model-View-Controller اختصار لـ MVC



MVC: مكونات

1. Model (النموذج) □

- يتعامل مع قاعدة البيانات
- (Business Logic) يحتوي على منطق الأعمال
- مثال: `User`, `Post`, `Product`

```
class Product extends Model
{
    public function getDiscountedPrice()
    {
        return $this->price * 0.9;
    }
}
```

2. View (العرض) □

- (HTML) واجهة المستخدم
- عرض البيانات للمستخدم
- `resources/views` في مجلد Blade ملفات

```
<h1>{{ $product->name }}</h1>
<p>السعر: {{ $product->price }}</p>
```

3. Controller (المتحكم) □

- Model و View يربط بين
- (Requests) يستقبل الطلبات
- يعالج المنطق
- (Response) يرجع الاستجابة

```
class ProductController extends Controller
{
    public function show($id)
    {
        $product = Product::find($id);
        return view('product.show', compact('product'));
    }
}
```

```
}
```

MVC؟ لماذا نستخدم

□ فصل الاهتمامات (Separation of Concerns)

- كل جزء له مسؤولية محددة

□ سهولة الصيانة

- تغيير واجهة المستخدم بدون تغيير منطق التطبيق

□ إعادة الاستخدام

- مختلفة Controllers في Model استخدام نفس

□ العمل الجماعي

- Controllers آخر على Views، مطور يعمل على

ما هي المتحكمات؟

التعريف

HTTP. لمعالجة طلبات methods يحتوي على **PHP** هو كلاس (Controller) المتحكم

قبل المتحكمات - المسارات المباشرة

```
// في routes/web.php
Route::get('/products', function () {
    $products = DB::table('products')->get();
    return view('products.index', compact('products'));
});

Route::get('/products/{id}', function ($id) {
    $product = DB::table('products')->find($id);
    return view('products.show', compact('product'));
});

Route::post('/products', function () {
    // سطر من الكود 50
```

```
DB::table('products')->insert([...]);
return redirect('/products');
});
```

المشاكل:

- الكود مكرر ❗
- صعوبة الصيانة ❗
- كبير جداً routes ملف ❗
- صعوبة إعادة الاستخدام ❗

بعد المتحكمات - حل منظم

```
// في routes/web.php
Route::get('/products', [ProductController::class, 'index']);
Route::get('/products/{id}', [ProductController::class, 'show']);
Route::post('/products', [ProductController::class, 'store']);
```

```
// في app/Http/Controllers/ProductController.php
class ProductController extends Controller
{
    public function index()
    {
        $products = Product::all();
        return view('products.index', compact('products'));
    }

    public function show($id)
    {
        $product = Product::find($id);
        return view('products.show', compact('product'));
    }

    public function store(Request $request)
    {
        Product::create($request->all());
        return redirect('/products');
    }
}
```

المميزات:

- كود منظم ونظيف ❗

- سهولة الصيانة والتطوير ٤
- Methods إعادة استخدام ٤
- سهولة الاختبار ٤

إنشاء المتحكمات

1. يدوياً إنشاء Controller

يمكنك إنشاء ملف في `app/Http/Controllers`:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class ProductController extends Controller
{
    public function index()
    {
        return 'قائمة المنتجات';
    }
}
```

2. (الطريقة الموصى بها) Artisan باستخدام Controller إنشاء

```
php artisan make:controller ProductController
```

هذا الأمر ينشئ ملف

```
app/Http/Controllers/ProductController.php
```

3. إنشاء Resource Controller

```
php artisan make:controller ProductController --resource
```

جاهزة methods مع 7 Controller ينشئ

- `index()` – عرض القائمة
- `create()` – نموذج الإنشاء
- `store()` – حفظ البيانات
- `show($id)` – عرض عنصر واحد
- `edit($id)` – نموذج التعديل
- `update($id)` – تحديث البيانات
- `destroy($id)` – حذف عنصر

4. إنشاء **Controller** مع **Model**

```
php artisan make:controller ProductController --resource --model=Product
```

تلقائياً `Model` مع استيراد `Controller` ينشئ.

5. إنشاء **API Controller**

```
php artisan make:controller API/ProductController --api
```

(لا يحتاجهما `API` لأن `edit()` و `create()` لكن بدون `Resource` مثل).

أنواع المتحكمات

1. Single Action Controller

واحد فقط `action` متحكم يحتوي على.

```
php artisan make:controller ShowProfileController --invokable
```

```
class ShowProfileController extends Controller
{
    public function __invoke($id)
    {
        $user = User::find($id);
        return view('profile', compact('user'));
    }
}
```

الاستخدام:

```
Route::get('/profile/{id}', ShowProfileController::class);
```

متى نستخدمه؟

- واحد معقد action عندما يكون لديك
- لفصل المنطق المعقد

2. Resource Controller

كاملة CRUD متحكم لإدارة.

```
class ProductController extends Controller
{
    // GET /products
    public function index()
    {
        $products = Product::all();
        return view('products.index', compact('products'));
    }

    // GET /products/create
    public function create()
    {
        return view('products.create');
    }

    // POST /products
    public function store(Request $request)
    {
        $validated = $request->validate([
            'name' => 'required|max:255',
            'price' => 'required|numeric',
        ]);

        Product::create($validated);
        return redirect()->route('products.index')
            ->with('success', 'تم إنشاء المنتج بنجاح');
    }

    // GET /products/{id}
    public function show(Product $product)
    {
        return view('products.show', compact('product'));
    }
}
```

```

}

// GET /products/{id}/edit
public function edit(Product $product)
{
    return view('products.edit', compact('product'));
}

// PUT/PATCH /products/{id}
public function update(Request $request, Product $product)
{
    $validated = $request->validate([
        'name' => 'required|max:255',
        'price' => 'required|numeric',
    ]);

    $product->update($validated);
    return redirect()->route('products.index')
        ->with('success', 'تم تحديث المنتج بنجاح');
}

// DELETE /products/{id}
public function destroy(Product $product)
{
    $product->delete();
    return redirect()->route('products.index')
        ->with('success', 'تم حذف المنتج بنجاح');
}
}

```

مع المسارات **Resource Controller** ربط

```
Route::resource('products', ProductController::class);
```

هذا السطر الواحد ينشئ 7 مسارات تلقائياً

```
php artisan route:list --name=products
```

Method	URI	Name	Action
GET	/products	products.index	index
GET	/products/create	products.create	create

Method	URI	Name	Action
POST	/products	products.store	store
GET	/products/{id}	products.show	show
GET	/products/{id}/edit	products.edit	edit
PUT/PATCH	/products/{id}	products.update	update
DELETE	/products/{id}	products.destroy	destroy

3. API Resource Controller

لـ APIs، بدون `create` و `edit`:

```
php artisan make:controller API/ProductController --api
```

```
Route::apiResource('products', ProductController::class);
```

ينشئ 5 مسارات فقط:

- `index`, `store`, `show`, `update`, `destroy`

4. Nested Resource Controllers

(للموارد المتداخلة) مثل تعليقات على مقال:

```
Route::resource('posts.comments', CommentController::class);
```

ينشئ مسارات مثل:

- `/posts/{post}/comments`
- `/posts/{post}/comments/{comment}`

```
class CommentController extends Controller
{
    public function index(Post $post)
    {
        return view('comments.index', [
            'post' => $post,
            'comments' => $post->comments
        ]);
    }
}
```

```

    });
}

public function store(Request $request, Post $post)
{
    $post->comments()->create($request->all());
    return redirect()->route('posts.comments.index', $post);
}
}

```

Resource Controllers

معيّنة فقط **Methods** تحديد

methods: إذا كنت لا تحتاج جميع الـ 7

```

// فقط index و show
Route::resource('products', ProductController::class)
    ->only(['index', 'show']);

// كل شيء ماعدا destroy
Route::resource('products', ProductController::class)
    ->except(['destroy']);

```

Parameters تسمية

```

Route::resource('products', ProductController::class)
    ->parameters([
        'products' => 'product_id'
    ]);

// النتيجة: /products/{product_id}

```

Resource Routes مع **Middleware**

```

Route::resource('products', ProductController::class)
    ->middleware('auth');

```

حقن التبعيات (Dependency Injection)

ما هو حقن التبعيات؟

تلقائياً Controller هو تقنية لإدخال الكائنات التي يحتاجها الـ **Dependency Injection**.

1. حقن Request

```
use Illuminate\Http\Request;

class ProductController extends Controller
{
    public function store(Request $request)
    {
        // تلقائياً Request object يحقن Laravel
        $name = $request->input('name');
        $price = $request->input('price');

        Product::create([
            'name' => $name,
            'price' => $price
        ]);

        return redirect()->route('products.index');
    }
}
```

2. حقن Model (Route Model Binding)

```
class ProductController extends Controller
{
    public function show(Product $product)
    {
        // Laravel يجلب المنتج من قاعدة البيانات تلقائياً
        return view('products.show', compact('product'));
    }

    public function update(Request $request, Product $product)
    {
        $product->update($request->all());
        return redirect()->route('products.show', $product);
    }
}
```

```

public function destroy(Product $product)
{
    $product->delete();
    return redirect()->route('products.index');
}
}

```

كيف يعمل؟

- تلقائياً `Product::find($id)` يبحث عن Laravel
- إذا لم يجد، يرجع 404 تلقائياً

3. حقن Services

```

use App\Services\PaymentService;

class OrderController extends Controller
{
    public function store(Request $request, PaymentService $payment)
    {
        // تلقائياً PaymentService يحقن Laravel
        $order = Order::create($request->all());
        $payment->process($order);

        return redirect()->route('orders.show', $order);
    }
}

```

4. حقن في Constructor

methods: لحقن تبعية تستخدم في جميع

```

class ProductController extends Controller
{
    protected $productService;

    public function __construct(ProductService $productService)
    {
        $this->productService = $productService;
    }

    public function index()
    {
        $products = $this->productService->getAllProducts();
    }
}

```

```

        return view('products.index', compact('products'));
    }

    public function store(Request $request)
    {
        $this->productService->createProduct($request->all());
        return redirect()->route('products.index');
    }
}

```

فوائد حقن التبعيات

□ فصل الاهتمامات (Separation of Concerns) □ Mock Objects كود نظيف وسهل القراءة □ سهولة الاختبار (يمكن استخدام إعادة الاستخدام) □

Middleware في المتحكمات

1. Middleware في Routes

```

Route::resource('products', ProductController::class)
    ->middleware('auth');

```

2. Middleware في Constructor

```

class ProductController extends Controller
{
    public function __construct()
    {
        // تحتاج تسجيل دخول كل methods
        $this->middleware('auth');

        // index و show ماعدا
        $this->middleware('auth')->except(['index', 'show']);

        // edit و store و create فقط
        $this->middleware('auth')->only(['create', 'store', 'edit']);
    }
}

```

3. Middleware methods لمخصص

```
class ProductController extends Controller
{
    public function __construct()
    {
        // التحقق من الصلاحيات للحذف والتحديث فقط
        $this->middleware('can:update,product')->only(['edit', 'update']);
        $this->middleware('can:delete,product')->only('destroy');
    }
}
```

أفضل الممارسات

أفعل:

1. استخدم أسماء واضحة.

```
// جيد
class ProductController extends Controller
class UserProfileController extends Controller
class AdminDashboardController extends Controller

// سيء
class MyController extends Controller
class Controller1 extends Controller
```

2. Controller لا تضع منطق معقد في

```
// سيء
public function store(Request $request)
{
    $product = new Product();
    $product->name = $request->name;
    $product->price = $request->price;
    $product->save();

    // إرسال إيميل
    Mail::to($user)->send(new ProductCreated($product));
}
```

```

// تحديث الإحصائيات
Statistics::increment('products_count');

// حفظ في الـ log
Log::info('Product created: ' . $product->id);

// سطر آخر 50...

return redirect()->route('products.index');
}

// جيد - استخدم Service Classes
public function store(Request $request, ProductService $service)
{
    $product = $service->createProduct($request->validated());
    return redirect()->route('products.index');
}

```

3. للتحقق Form Requests استخدم

```

// سيء
public function store(Request $request)
{
    $request->validate([
        'name' => 'required|max:255',
        'price' => 'required|numeric|min:0',
        'description' => 'required',
    ]);
    // قاعدة أخرى... 20

    ;([
    {

// جيد
public function store(StoreProductRequest $request)
{
    // التحقق يتم تلقائياً
    Product::create($request->validated());
}

```

إنشاء Form Request:

```
php artisan make:request StoreProductRequest
```

```

class StoreProductRequest extends FormRequest
{

```

```

public function rules()
{
    return [
        'name' => 'required|max:255',
        'price' => 'required|numeric|min:0',
        'description' => 'required',
    ];
}
}

```

4. **Route Model Binding** استخدم

```

// سيء ❌
public function show($id)
{
    $product = Product::find($id);
    if (!$product) {
        abort(404);
    }
    return view('products.show', compact('product'));
}

// جيد ✅
public function show(Product $product)
{
    return view('products.show', compact('product'));
}

```

5. **Array أو compact()** استخدم

```

// جيد ✅
return view('products.index', compact('products', 'categories'));

// جيد أيضاً ✅
return view('products.index', [
    'products' => $products,
    'categories' => $categories
]);

// قديم (لكن يعمل) ❌
return view('products.index')->with('products', $products);

```


❗ لا تفعل:

1. (DRY Principle) لا تكرر الكود

```
// ❗ سيء
public function adminIndex()
{
    $products = Product::where('status', 'active')->get();
    return view('admin.products', compact('products'));
}

public function userIndex()
{
    $products = Product::where('status', 'active')->get();
    return view('user.products', compact('products'));
}

// ❗ مشترك method جيد - استخدم
protected function getActiveProducts()
{
    return Product::where('status', 'active')->get();
}

public function adminIndex()
{
    $products = $this->getActiveProducts();
    return view('admin.products', compact('products'));
}

public function userIndex()
{
    $products = $this->getActiveProducts();
    return view('user.products', compact('products'));
}
```

2. Controller معقدة في queries لا تضع

```
// ❗ سيء
public function index()
{
    $products = DB::table('products')
        ->join('categories', 'products.category_id', '=', 'categories.id')
        ->join('brands', 'products.brand_id', '=', 'brands.id')
        ->select('products.*', 'categories.name as category_name')
        ->where('products.status', 'active')
```

```

->orderBy('products.created_at', 'desc')
->paginate(10);

return view('products.index', compact('products'));
}

// ؟ جيد - استخدم Query Scopes أو Repository جيد
public function index(ProductRepository $repository)
{
    $products = $repository->getActiveWithRelations();
    return view('products.index', compact('products'));
}

```

التمارين العملية

؟ بسيط Controller تمرين 1: إنشاء

```
php artisan make:controller PageController
```

```

class PageController extends Controller
{
    public function home()
    {
        return view('home');
    }

    public function about()
    {
        return view('about');
    }

    public function contact()
    {
        return view('contact');
    }
}

```

المسارات:

```

Route::get('/', [PageController::class, 'home'])->name('home');
Route::get('/about', [PageController::class, 'about'])->name('about');

```

```
Route::get('/contact', [PageController::class, 'contact'])->name('contact');
```

تمرين 2: Resource Controller

```
php artisan make:controller ProductController --resource
```

المسارات:

```
Route::resource('products', ProductController::class);
```

التطبيق في **ProductController:**

```
class ProductController extends Controller
{
    public function index()
    {
        $products = [
            ['id' => 1, 'name' => 'لابتوب', 'price' => 5000],
            ['id' => 2, 'name' => 'هاتف', 'price' => 3000],
            ['id' => 3, 'name' => 'تابلت', 'price' => 2000],
        ];

        return view('products.index', compact('products'));
    }

    public function show($id)
    {
        $products = [
            1 => ['id' => 1, 'name' => 'لابتوب', 'price' => 5000],
            2 => ['id' => 2, 'name' => 'هاتف', 'price' => 3000],
            3 => ['id' => 3, 'name' => 'تابلت', 'price' => 2000],
        ];

        $product = $products[$id] ?? abort(404);

        return view('products.show', compact('product'));
    }

    public function create()
    {
        return view('products.create');
    }

    public function store(Request $request)
```

```

{
    // في التطبيق الحقيقي، نحفظ في قاعدة البيانات
    return redirect()->route('products.index')
        ->with('success', 'تم إضافة المنتج بنجاح');
}
}

```

تمرين 3: Single Action Controller

```
php artisan make:controller ShowDashboardController --invokable
```

```

class ShowDashboardController extends Controller
{
    public function __invoke()
    {
        $stats = [
            'users' => 150,
            'products' => 80,
            'orders' => 320,
            'revenue' => 45000
        ];

        return view('dashboard', compact('stats'));
    }
}

```

المسار:

```

Route::get('/dashboard', ShowDashboardController::class)
    ->name('dashboard');

```

تمرين 4: Controller مع Dependency Injection

```
php artisan make:controller UserController
```

```

use Illuminate\Http\Request;

class UserController extends Controller
{
    public function store(Request $request)
    {
        // التحقق من البيانات
    }
}

```

```

        $validated = $request->validate([
            'name' => 'required|max:255',
            'email' => 'required|email|unique:users',
            'password' => 'required|min:8',
        ]);

        // في التطبيق الحقيقي: User::create($validated)

        return redirect()->route('users.index')
            ->with('success', 'تم إنشاء المستخدم بنجاح');
    }

    public function update(Request $request, $id)
    {
        $validated = $request->validate([
            'name' => 'required|max:255',
            'email' => 'required|email',
        ]);

        // في التطبيق الحقيقي: $user->update($validated)

        return redirect()->route('users.show', $id)
            ->with('success', 'تم تحديث المستخدم بنجاح');
    }
}

```

5 تمرين: Middleware في Controller

```

class AdminController extends Controller
{
    public function __construct()
    {
        // تحتاج تسجيل دخول كل methods
        $this->middleware('auth');
    }

    public function dashboard()
    {
        return view('admin.dashboard');
    }

    public function users()
    {
        return view('admin.users');
    }
}

```

```
public function settings()
{
    return view('admin.settings');
}
}
```

المسارات:

```
Route::prefix('admin')->name('admin.')->group(function () {
    Route::get('/dashboard', [AdminController::class, 'dashboard'])
        ->name('dashboard');
    Route::get('/users', [AdminController::class, 'users'])
        ->name('users');
    Route::get('/settings', [AdminController::class, 'settings'])
        ->name('settings');
});
```

كامل CRUD التحدي: نظام

مع (Posts) لإدارة المقالات CRUD قم بإنشاء نظام

1. إنشاء Controller:

```
php artisan make:controller PostController --resource
```

2. تعريف المسارات:

```
Route::resource('posts', PostController::class);
```

3. تطبيق Methods:

```
class PostController extends Controller
{
    // عرض جميع المقالات
    public function index()
    {
        $posts = [
            ['id' => 1, 'title' => 'مقدمة في Laravel', 'author' => 'أحمد'],
            ['id' => 2, 'title' => 'Controllers تعلم', 'author' => 'محمد'],
        ];
    }
}
```

```

];
return view('posts.index', compact('posts'));
}

// عرض نموذج إنشاء مقال جديد
public function create()
{
    return view('posts.create');
}

// حفظ المقال الجديد
public function store(Request $request)
{
    $validated = $request->validate([
        'title' => 'required|max:255',
        'content' => 'required',
    ]);

    // Post::create($validated);

    return redirect()->route('posts.index')
        ->with('success', 'تم إنشاء المقال بنجاح!');
}

// عرض مقال واحد
public function show($id)
{
    $post = [
        'id' => $id,
        'title' => 'مقدمة Laravel',
        'content' => 'محتوى المقال...',
        'author' => 'أحمد'
    ];
    return view('posts.show', compact('post'));
}

// عرض نموذج التعديل
public function edit($id)
{
    $post = [
        'id' => $id,
        'title' => 'مقدمة Laravel',
        'content' => 'محتوى المقال...'
    ];
    return view('posts.edit', compact('post'));
}

// تحديث المقال

```

```

public function update(Request $request, $id)
{
    $validated = $request->validate([
        'title' => 'required|max:255',
        'content' => 'required',
    ]);

    // $post->update($validated);

    return redirect()->route('posts.show', $id)
        ->with('success', 'تم تحديث المقال بنجاح!');
}

// حذف المقال
public function destroy($id)
{
    // $post->delete();

    return redirect()->route('posts.index')
        ->with('success', 'تم حذف المقال بنجاح');
}
}

```

ملخص ؟

في هذا الدرس، تعلمت:

Controllers بطرق مختلفة ؟ أنواع Controllers وأهميته ؟ ما هي المتحكمات ولماذا نستخدمها ؟ إنشاء MVC نمط ؟ حقن التبعية ؟ methods والـ 7 Resource Controllers ؟ أفضل الممارسات ؟ Controllers في Middleware استخدام ؟ (Dependency Injection)

مصادر إضافية ؟

- [Laravel Controllers Documentation](#)
- [Resource Controllers](#)
- [Dependency Injection](#)

اختبر نفسك ؟

قبل الانتقال للدرس التالي، تأكد من إجابتك على

1. Model و View و Controller ما الفرق بين
2. routes بدلاً من وضع الكود في Controllers ما فائدة استخدام
3. API Resource Controller و Resource Controller ما الفرق بين
4. (Dependency Injection) ما هو حقن التبعية
5. Middleware في Constructor كيف تستخدم

الدرس التالي

وواجهات المستخدم **Blade Templates**: جاهز للمزيد؟ انتقل إلى **الدرس 4**

في الدرس 4، ستتعلم

- Blade محرك القوالب
- التوجيهات (Directives)
- المكونات (Components)
- التخطيطات (Layouts)
- والمزيد!

؟ تعلم سعيد